Final year project

Name :- Saud Masaud Alam

Roll number :- U22CS094

Project Description

This project focuses on the automated classification of plant leaf diseases using deep learning techniques. Leveraging the publicly available PlantDoc dataset, which consists of labeled images of leaves from various plant species affected by different diseases, the goal is to develop a highly accurate image classification model that can assist in early disease detection and management in agriculture.

Using state-of-the-art deep learning models (such as Xception and EfficientNetB0), the project implements a pipeline for:

- Loading and preprocessing leaf images,
- Organizing the dataset into training, validation, and test sets,
- Applying transfer learning with pre-trained convolutional neural networks,
- Training and optimizing the model using techniques like early stopping, learning rate scheduling, and class weighting (to address class imbalance),
- Evaluating model performance with metrics suited for multiclass classification (accuracy, precision, recall, weighted F1-score, and detailed per-class reports),
- Visualizing results through confusion matrices and per-class score graphs.

The primary objective is to accurately distinguish between healthy and diseased leaves across multiple crop types, helping farmers and agronomists to rapidly identify plant health issues using AI-powered tools. Project outcomes include a trained model, validation of its accuracy on held-out data, visualizations of classification results, and recommendations for improving deployment-ready performance in real-world settings.

Xception Model Overview

Xception (Extreme Inception) is a deep convolutional neural network architecture developed by François Chollet, the creator of Keras. Introduced in 2017, Xception is an improvement over the Inception architecture and stands out due to its use of depthwise separable convolutions. This architectural change results in a model that is more efficient, has fewer parameters, and achieves higher accuracy on many visual recognition tasks.

Key Features of Xception:

- Depthwise Separable Convolutions:
  Traditional convolution layers combine spatial filtering and feature combination in a single step. Xception separates these into two steps—a depthwise convolution (spatial filtering independently for each channel) followed by a pointwise (1x1) convolution (combines these features across channels). This makes the model both computationally efficient and flexible.
- Pre-trained Weights:
  In Keras, the Xception model comes with weights pre-trained on the ImageNet dataset. This allows transfer learning—using knowledge from millions of general images to jump-start your specific plant disease recognition task.
- Input Size:
  Standard input dimensions are 299x299x3 (height, width, RGB channels). Images must be resized and preprocessed accordingly.

Why Use Xception?

- Strong Baseline Performance: Xception achieves accuracy comparable to even larger models (like Inception-v3 and ResNet) with lower computational costs.
- Good for Fine-grained Recognition: Its architectural design allows learning subtle differences in visual features—a must for distinguishing between similar plant diseases.
- Ease of Use: Available directly via Keras, making it suitable for research and production.

How Xception is Used in Your Project

1. Transfer Learning:
   You load the Xception base model with pre-trained ImageNet weights, omitting the top (final classification) layers.

2. Custom Classification Head:
   A new classifier (dense layers and dropout) is added on top to adapt the model to your specific number of disease classes.
3. Feature Extraction Training:
   During initial training, the pre-trained Xception layers are “frozen” (weights not updated) to leverage already learned features. Only the custom classification head is trained.
4. Advanced Fine-tuning (optional):
   For further improvement, you can unfreeze some Xception layers later and train them on your leaf dataset at a lower learning rate.
5. Image Preprocessing:
   Input images are resized to 299×299 and pixel values are scaled to the range [-1, 1] using `preprocess_input` from `tf.keras.applications.xception`.
6. Multiclass Output:
   The final dense layer uses softmax activation to output probabilities for each class (disease type).

Step1:- load dataset from drive .
Code :-

```
import os
import numpy as np
import cv2  # for image loading and resizing


def load_plantdoc_data(data_dir, img_size=(299, 299)):
    X = []
    y = []
    class_names = sorted(os.listdir(data_dir))
    class_to_idx = {cls_name: idx for idx, cls_name in
enumerate(class_names)}

    for cls_name in class_names:
        cls_dir = os.path.join(data_dir, cls_name)
        if not os.path.isdir(cls_dir):
            continue
        for img_name in os.listdir(cls_dir):
            img_path = os.path.join(cls_dir, img_name)
            img = cv2.imread(img_path)
```

```
            if img is None:
                continue
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  # convert to
RGB
            img = cv2.resize(img, img_size)
            X.append(img)
            y.append(class_to_idx[cls_name])
    X = np.array(X, dtype=np.float32)
    y = np.array(y, dtype=np.int32)
    return X, y, class_names

from google.colab import drive

# Step 1: Mount Google Drive (only once)
drive.mount('/content/drive')

# Step 2: Correct path to your dataset folder inside Google Drive
dataset_root = '/content/drive/MyDrive/PlantDoc-Dataset'  # Corrected
path
train_dir = f'{dataset_root}/train'
test_dir = f'{dataset_root}/test'

# Step 3: Call your loading function correctly
X_train, y_train, class_names = load_plantdoc_data(train_dir)
X_test, y_test, _ = load_plantdoc_data(test_dir)
```

Output:-

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Step 2: Normalize Images

Code:-

```
def preprocess_images(X):
    """
    Normalize images to [0,1] range
    """
    return X / 255.0

X_train = preprocess_images(X_train)
X_test = preprocess_images(X_test)
```

```
Step 3: Create Validation Dataset from Training Data
```

Code:-

```
from sklearn.model_selection import train_test_split

def create_validation_split(X_train, y_train, val_size=0.2):
    """
    Split training data into train and validation sets with
stratification
    """
    return train_test_split(
        X_train, y_train, test_size=val_size, stratify=y_train,
random_state=42
    )

X_train, X_val, y_train, y_val = create_validation_split(X_train,
y_train, val_size=0.2)
```

```
Step 4: Convert Labels to One-Hot Encoding
```

Code:-

```
from tensorflow.keras.utils import to_categorical

num_classes = len(class_names)
y_train_cat = to_categorical(y_train, num_classes)
y_val_cat = to_categorical(y_val, num_classes)
y_test_cat = to_categorical(y_test, num_classes)
```

```
Step 5: Build a Keras Model (Xception)
```

Code:-

```
import tensorflow as tf
from tensorflow.keras.applications import Xception
from tensorflow.keras import layers, models, optimizers

def build_xception_model(num_classes, input_shape=(299, 299, 3)):
    # Load the pre-trained Xception model without the top
classification layer
    base_model = Xception(include_top=False, weights='imagenet',
input_shape=input_shape)
    base_model.trainable = False  # Freeze base model layers
```

```
    # Build your custom classification head
    model = models.Sequential([
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dropout(0.3),
        layers.Dense(512, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.3),
        layers.Dense(num_classes, activation='softmax')
    ])

    # Compile the model
    model.compile(
        optimizer=optimizers.Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

# Example usage:
model = build_xception_model(num_classes=27)  # Adjust num_classes to
your dataset
```

```
Step 6: Train the Model with Early Stopping and Learning Rate Reduction
```

Code:-

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

early_stop = EarlyStopping(monitor='val_accuracy', patience=5,
restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', patience=3,
factor=0.5, min_lr=1e-7)

history = model.fit(
    X_train, y_train_cat,
    validation_data=(X_val, y_val_cat),
    epochs=20,
    batch_size=32,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

Output:-

```
Epoch 1/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 971s 17s/step - accuracy: 0.2398 - loss: 3.0052 - val_accuracy: 0.3944 - val_loss: 2.1239 - learning_rate: 0.0010
Epoch 2/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 950s 16s/step - accuracy: 0.5492 - loss: 1.4432 - val_accuracy: 0.4892 - val_loss: 1.7103 - learning_rate: 0.0010
Epoch 3/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 989s 17s/step - accuracy: 0.6629 - loss: 1.0602 - val_accuracy: 0.5108 - val_loss: 1.5842 - learning_rate: 0.0010
Epoch 4/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 980s 17s/step - accuracy: 0.7467 - loss: 0.8216 - val_accuracy: 0.5151 - val_loss: 1.5408 - learning_rate: 0.0010
Epoch 5/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 942s 16s/step - accuracy: 0.7816 - loss: 0.7324 - val_accuracy: 0.5172 - val_loss: 1.5561 - learning_rate: 0.0010
Epoch 6/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 982s 16s/step - accuracy: 0.7875 - loss: 0.6560 - val_accuracy: 0.5129 - val_loss: 1.5689 - learning_rate: 0.0010
Epoch 7/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 983s 16s/step - accuracy: 0.8291 - loss: 0.5232 - val_accuracy: 0.5409 - val_loss: 1.5766 - learning_rate: 0.0010
Epoch 8/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 959s 17s/step - accuracy: 0.8305 - loss: 0.5079 - val_accuracy: 0.5625 - val_loss: 1.4984 - learning_rate: 5.0000e-04
Epoch 9/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 975s 16s/step - accuracy: 0.8576 - loss: 0.4245 - val_accuracy: 0.5496 - val_loss: 1.5120 - learning_rate: 5.0000e-04
Epoch 10/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 954s 16s/step - accuracy: 0.8886 - loss: 0.3655 - val_accuracy: 0.5453 - val_loss: 1.5091 - learning_rate: 5.0000e-04
```

```
Epoch 11/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 1003s 16s/step - accuracy: 0.8823 - loss: 0.3605 - val_accuracy: 0.5690 - val_loss: 1.5175 - learning_rate: 5.0000e-04
Epoch 12/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 974s 16s/step - accuracy: 0.9062 - loss: 0.3258 - val_accuracy: 0.5539 - val_loss: 1.5390 - learning_rate: 2.5000e-04
Epoch 13/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 971s 16s/step - accuracy: 0.9175 - loss: 0.2785 - val_accuracy: 0.5690 - val_loss: 1.5255 - learning_rate: 2.5000e-04
Epoch 14/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 996s 16s/step - accuracy: 0.9346 - loss: 0.2577 - val_accuracy: 0.5668 - val_loss: 1.5433 - learning_rate: 2.5000e-04
Epoch 15/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 942s 16s/step - accuracy: 0.9229 - loss: 0.2782 - val_accuracy: 0.5625 - val_loss: 1.5378 - learning_rate: 1.2500e-04
Epoch 16/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 919s 16s/step - accuracy: 0.9257 - loss: 0.2643 - val_accuracy: 0.5711 - val_loss: 1.5371 - learning_rate: 1.2500e-04
Epoch 17/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 949s 16s/step - accuracy: 0.9256 - loss: 0.2532 - val_accuracy: 0.5625 - val_loss: 1.5508 - learning_rate: 1.2500e-04
Epoch 18/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 959s 16s/step - accuracy: 0.9342 - loss: 0.2430 - val_accuracy: 0.5690 - val_loss: 1.5573 - learning_rate: 6.2500e-05
Epoch 19/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 925s 16s/step - accuracy: 0.9405 - loss: 0.2214 - val_accuracy: 0.5711 - val_loss: 1.5621 - learning_rate: 6.2500e-05
Epoch 20/20
58/58 ━━━━━━━━━━━━━━━━━━━━ 945s 16s/step - accuracy: 0.9353 - loss: 0.2358 - val_accuracy: 0.5668 - val_loss: 1.5684 - learning_rate: 6.2500e-05
```

```
Step 7: Evaluate the Model on Test Data
```

Code:-

```
from sklearn.metrics import accuracy_score, f1_score,
classification_report

y_pred_proba = model.predict(X_test)
y_pred = y_pred_proba.argmax(axis=1)
y_true = y_test_cat.argmax(axis=1)

acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred, average='weighted')
report = classification_report(y_true, y_pred,
target_names=class_names)

print(f"Test Accuracy: {acc:.4f}")
print(f"Test Weighted F1 Score: {f1:.4f}")
print("Classification Report:\n", report)
```

Output:-

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 99s 12s/step
Test Accuracy: 0.5042
Test Weighted F1 Score: 0.4997
Classification Report:
                              precision    recall  f1-score   support

           Apple Scab Leaf       0.44      0.80      0.57        10
                Apple leaf       0.50      0.67      0.57         9
           Apple rust leaf       0.75      0.60      0.67        10
          Bell_pepper leaf       0.50      0.62      0.56         8
     Bell_pepper leaf spot       0.20      0.11      0.14         9
            Blueberry leaf       0.25      0.18      0.21        11
               Cherry leaf       0.60      0.30      0.40        10
       Corn Gray leaf spot       0.17      0.50      0.25         4
          Corn leaf blight       0.80      0.33      0.47        12
            Corn rust leaf       0.78      0.70      0.74        10
                Peach leaf       0.67      0.67      0.67         9
  Potato leaf early blight       0.27      0.38      0.32         8
   Potato leaf late blight       0.29      0.25      0.27         8
            Raspberry leaf       0.75      0.86      0.80         7
             Soyabean leaf       0.44      0.50      0.47         8
Squash Powdery mildew leaf       0.71      0.83      0.77         6
           Strawberry leaf       0.80      1.00      0.89         8
  Tomato Early blight leaf       0.62      0.56      0.59         9
 Tomato Septoria leaf spot       0.22      0.36      0.28        11
               Tomato leaf       0.75      0.38      0.50         8
Tomato leaf bacterial spot       0.25      0.22      0.24         9
   Tomato leaf late blight       0.57      0.40      0.47        10
  Tomato leaf mosaic virus       0.00      0.00      0.00        10
  Tomato leaf yellow virus       0.83      0.83      0.83         6
          Tomato mold leaf       0.13      0.33      0.19         6
                grape leaf       0.91      0.83      0.87        12
      grape leaf black rot       1.00      0.75      0.86         8

                  accuracy                           0.50       236
                 macro avg       0.53      0.52      0.50       236
              weighted avg       0.53      0.50      0.50       236

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

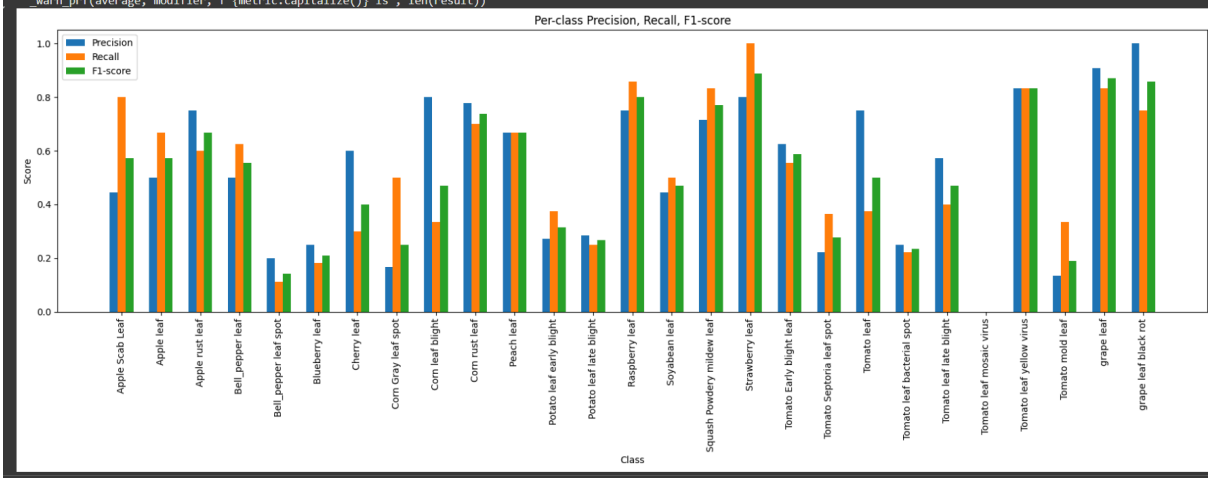
Step8:- graph of this result .

Code ➖

```
from sklearn.metrics import precision_recall_fscore_support
import numpy as np
```

```
precision, recall, f1, _ = precision_recall_fscore_support(y_true,
y_pred, labels=np.arange(len(class_names)))

x = np.arange(len(class_names))
plt.figure(figsize=(18,7))
plt.bar(x-0.2, precision, width=0.2, label='Precision')
plt.bar(x, recall, width=0.2, label='Recall')
plt.bar(x+0.2, f1, width=0.2, label='F1-score')
plt.xticks(x, class_names, rotation=90)
plt.xlabel("Class")
plt.ylabel("Score")
plt.title("Per-class Precision, Recall, F1-score")
plt.legend()
plt.tight_layout()
plt.show()
```

Output:-

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```